



AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

FACULTY OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ADVANCED PROGRAMMING IN WEB TECHNOLOGY

PROJECT PROPOSAL

*LiquidityOS: A SaaS Application for Smart Liquidity Management in Mobile Financial Service
(MFS) Agent Networks*

Section: C
Semester: Summer 2025-26

Name	ID
MD. Muinul Islam	23-53338-3
Md. Mohaiminul Haque	23-50302-1

Supervised By

MD. KHAIRUL ALAM MAZUMDER

1. Executive Summary

LiquidityOS is a SaaS-based liquidity coordination platform for mobile financial service networks. It acts as the operating layer between agents, liquidity coordinators, and MFS providers so that cash shortages, e-money shortages, and coordination delays can be detected, matched, settled, and audited in one system.

The platform is designed for networks where service continuity depends on fast liquidity movement. Instead of relying on phone calls, manual follow-ups, or informal trust, LiquidityOS creates a structured lifecycle: a participant registers, gets verified, opens a request, receives offers, accepts an assignment, completes a wallet transfer, and earns or loses points based on the outcome.

From a business perspective, the platform reduces failed customer transactions, shortens the time needed to source liquidity, improves provider visibility across areas, and creates incentives for good liquidity behavior. From an operational perspective, it gives every actor a clear workflow, a controlled permission model, and a full audit trail.

What the platform solves	How it solves it
Liquidity discovery	Matches shortage requests to nearby coordinators, agents, or provider-controlled wallets
Liquidity execution	Uses wallet-to-wallet settlement with atomic balance updates and idempotency
Network coordination	Lets providers see shortages, approve coordinators, and monitor fill rates
Accountability	Records every request, offer, assignment, transfer, reversal, and point event
Incentives	Awards reward points for successful liquidity support and can reverse them when needed

2. Problem Statement

Mobile financial service networks break down when liquidity is unevenly distributed. An agent may be active, trusted, and busy, but still unable to serve a customer because the outlet is short on physical cash or short on e-money. That failure is not usually a demand problem. It is a working-capital problem.

The difficulty is that liquidity lives in many places at once. One outlet may hold cash, another may hold e-money, and a coordinator may hold reserve balances across multiple provider relationships. Without a shared coordination layer, those balances stay fragmented, invisible, and underused even when nearby outlets are struggling.

2.1 Physical Cash Shortage

This is the classic cash-out failure. The agent may have enough provider wallet balance to support the transaction, but the outlet cannot physically pay the customer. In practice, the agent must refuse the service, delay the customer, or source cash informally from another party.

Example: Agent A has ₦100,000 in bKash balance and only ₦5,000 in physical cash. A customer wants to withdraw ₦50,000. Even though the digital balance is available, the agent cannot complete the cash payout because the physical cash drawer cannot cover the request.

2.2 E-Money Shortage

The opposite failure also happens. The outlet may have enough physical cash to accept deposits, but the digital wallet balance is too low to settle a cash-in or wallet-based transaction. That makes the outlet look open, but it is effectively unable to process the customer's request.

Example: Agent B has ₦200,000 in cash but only ₦3,000 in provider wallet balance, so the agent cannot complete a cash-in transaction. The service window is technically open, but the transactional capacity is blocked.

2.3 Existing Operational Gaps

- There is no central place where shortages, surpluses, and matching status are visible together.
- Agents usually need to call people they know instead of discovering the best nearby liquidity source in the system.
- Manual coordination creates delays, which means the customer sees a failed or postponed service even when liquidity exists nearby.
- Providers cannot easily see where shortages are clustering, which areas are repeatedly failing, or which coordinators are most effective.
- Without rewards and accountability, participants have little motivation to move liquidity proactively or respond quickly to network demand.
- When transfers are handled outside a system, reconciliation becomes difficult and disputes become expensive to resolve.

3. Proposed Solution

LiquidityOS turns liquidity movement into a managed workflow instead of an informal side conversation. The system lets a shortage become a structured request, lets qualified participants submit offers, lets the appropriate authority assign the request, and then executes the money movement through wallet-led settlement.

3.1 Liquidity Request System

Any authorized actor can create a liquidity request with enough context for matching: amount, liquidity type, provider scope, area, urgency, and expiration time. That context is what allows the platform to search, rank, and route the request instead of simply broadcasting it everywhere.

3.2 Matching and Assignment

Once a request is open, the platform exposes it only to actors who are allowed to see it. Coordinators can submit offers with available amounts and estimated arrival times. The request owner, or the authorized provider/controller, then selects the best offer and locks the request into assignment so that two parties do not claim the same shortage.

3.3 Transfer Execution

After assignment, LiquidityOS executes the real value movement between wallets. The transfer is atomic, so debit and credit either both happen or neither happens. The transfer is idempotent, so network retries do not duplicate money movement. If something goes wrong, the workflow can be failed, reversed, or disputed without losing traceability.

3.4 Incentives and Monitoring

Every successful liquidity activity can create point transactions, which gives the network a reason to keep balances available instead of waiting until a shortage becomes urgent. Providers and administrators also get dashboards and reports that show fill rate, delay, hotspot concentration, and actor performance so they can tune the network proactively.

Solution layer	What it does
Identity layer	Registers, verifies, and authenticates every participant
Coordination layer	Creates requests, offers, assignments, and approvals
Settlement layer	Moves value across wallets in an auditable way
Incentive layer	Tracks reward points and compensating reversals
Control layer	Provides dashboards, restrictions, audits, and disputes

4. System Architecture

The implementation is centered on NestJS as the application framework, PostgreSQL as the system of record, TypeORM as the persistence layer, and a mailer layer for OTP and operational notifications. NestJS is a strong fit because the domain is modular, stateful, and workflow-heavy, with clear boundaries around authentication, requests, matching, wallets, transfers, and reporting.

PostgreSQL stores the authoritative business data. TypeORM maps the domain entities to relational tables and supports the transaction boundaries required for wallet movement, request assignment, point adjustments, and audit logging. The mailer layer handles email-based OTP delivery, password reset emails, and transactional notifications.

Layer	Chosen technology	Why it fits LiquidityOS
API and application layer	NestJS	Modular architecture, guards, interceptors, pipes, schedulers, and easy separation of concerns
Persistence layer	TypeORM	Entity-based modeling, repository patterns, transaction support, and migration workflows
Database	PostgreSQL	Strong relational integrity, locking, transactional consistency, and mature indexing
Email and notification layer	NestJS MailerModule / SMTP or	Sends OTP, password reset, request, assignment, transfer, and dispute notifications

	transactional email provider	
Validation layer	class-validator and class-transformer	DTO validation for all API contracts
Documentation layer	Swagger / OpenAPI	Clear API contract for frontend and integration consumers

4.1 Suggested NestJS Project Structure

src/
├── auth/
├── users/
├── providers/
├── agents/
├── coordinators/
├── wallets/
├── liquidity-requests/
├── liquidity-offers/
├── liquidity-transfers/
├── points/
├── notifications/
├── mailer/
├── disputes/
├── audit-logs/
├── common/
├── database/
└── config/

Each module should contain controllers, services, DTOs, entities, and guards where needed. Shared cross-cutting concerns such as pagination, response envelopes, role guards, tenant resolution, and email template helpers should live in `common` or `mailer`.

4.2 Email Notification Triggers

Action	Trigger timing	Recipient	Subject line	Content
User registers	After successful registration	User	Verify Your LiquidityOS Account	OTP code, expiry time, welcome message
Password reset requested	After /auth/forgot-password	User	Reset Your Password	Reset link/OTP, expiry time
Liquidity request created	After request enters OPEN state	Nearby coordinators/providers	New Liquidity Request Available	Request type, amount, area, urgency

Offer accepted	After assignment created	Coordinator & requester	Offer Accepted – [Request ID]	Assigned amount, ETA, counterparty
Transfer completed	After transfer status = SUCCESS	Both wallet owners	Transfer Completed – [Amount]	Amount, timestamp, reference ID
Dispute opened	After dispute created	Admin & involved parties	Dispute Opened – [Request ID]	Reason, involved transfer/request

4.3 Technical Requirements Summary

Category	Requirement
Language	TypeScript
Framework	NestJS (latest stable)
Authentication	JWT + @nestjs/passport + Role & Tenant Guards
Database & ORM	PostgreSQL + TypeORM
Relations	@OneToMany / @ManyToOne / @ManyToMany + @JoinTable
Input Validation	class-validator + class-transformer + global ValidationPipe
Documentation	@nestjs/swagger with descriptions/examples
Email Integration	NestJS MailerModule + SMTP + templates
Password Security	bcryptjs hashing
Idempotency	Idempotency-Key header on write endpoints
Error Handling	HttpException + global exception filter
Project Structure	Feature modules per Section 4.1
Testing	Jest — auth, transfer atomicity, matching logic
Configuration	@nestjs/config + .env
Version Control	Git + GitHub with commit history + README

5. System Actors and Responsibilities

Actor	Responsibilities	Can Do	Cannot Do
System Admin	Platform governance, user control, disputes, monitoring	Suspend users, approve or review records, manage policies	Provide liquidity or participate in transfers as a business actor
MFS Provider	Owns and oversees a provider network	Recruit agents, approve coordinators, supply liquidity, monitor shortages	Act outside approved tenant scope
Agent	Customer-facing MFS outlet	Request liquidity, maintain wallets, supply excess liquidity when allowed	Fulfill another agent's request directly

Liquidity Coordinator	Balances liquidity across the network	Submit offers, request liquidity, transfer funds, earn points	Bypass wallet or tenant rules
-----------------------	---------------------------------------	---	-------------------------------

5.1 Permission Summary

Capability	Admin	Provider	Agent	Coordinator
Register account	Yes	Yes	Yes	Yes
Verify identity	Yes	Yes	Yes	Yes
Create request	No	Yes	Yes	Yes
Submit offer	No	No	No	Yes
Assign request	Yes	Yes	Limited	Limited
Execute transfer	No	Yes	Yes	Yes
View dashboards	Yes	Yes	Own scope	Own scope
Resolve disputes	Yes	No	No	No

6. Database Entities

The brief names the primary data objects, but a working system also needs support entities for verification, notifications, disputes, and auditability. The tables below give the full field-level schema for each entity, followed by the complete set of relationships between them.

6.1 Entity Field Reference

User - *authentication identity for all actors*

Field	Type	Key	Notes
id	UUID	PK	Primary key
name	varchar(120)		Full name
email	varchar(150)	Unique	Login identifier
phone	varchar(20)	Unique, Nullable	Alternate login identifier
password_hash	varchar(255)		bcrypt hash
role	enum		ADMIN, PROVIDER, AGENT, COORDINATOR
status	enum		ACTIVE, SUSPENDED, PENDING
created_at	timestamp		Record creation time
updated_at	timestamp		Last update time

Provider - *MFS organization / tenant*

Field	Type	Key	Notes
id	UUID	PK	Primary key
name	varchar(150)		Organization name
tenant_code	varchar(30)	Unique	Tenant scoping key
status	enum		PENDING, APPROVED, REJECTED, SUSPENDED
created_at	timestamp		Record creation time
updated_at	timestamp		Last update time

Area - *geographic grouping used for matching and reporting*

Field	Type	Key	Notes
id	UUID	PK	Primary key
name	varchar(100)		Display name
code	varchar(20)	Unique	Short area code
status	enum		ACTIVE, INACTIVE

Agent - *provider outlet; belongs to exactly one provider*

Field	Type	Key	Notes
id	UUID	PK	Primary key
user_id	UUID	FK → User.id, Unique	Owning identity
provider_id	UUID	FK → Provider.id	Owning provider
area_id	UUID	FK → Area.id	Outlet location
shop_name	varchar(150)		Outlet display name
address	varchar(255)		Physical address
status	enum		ACTIVE, SUSPENDED

LiquidityCoordinator - *independent liquidity actor; can link to multiple providers*

Field	Type	Key	Notes
id	UUID	PK	Primary key
user_id	UUID	FK → User.id, Unique	Owning identity
area_id	UUID	FK → Area.id	Operating area
name	varchar(150)		Display name
status	enum		ACTIVE, SUSPENDED

CoordinatorProvider - *join table representing provider approval of a coordinator*

Field	Type	Key	Notes
-------	------	-----	-------

id	UUID	PK	Primary key
coordinator_id	UUID	FK → LiquidityCoordinator.id	Coordinator side
provider_id	UUID	FK → Provider.id	Provider side
status	enum		PENDING, APPROVED, REJECTED
approved_at	timestamp	Nullable	Set when approved
—	—	Unique (coordinator_id, provider_id)	Prevents duplicate links

Wallet - *value container; one balance per wallet type*

Field	Type	Key	Notes
id	UUID	PK	Primary key
owner_id	UUID	FK (polymorphic) → Agent.id / LiquidityCoordinator.id	Wallet holder
owner_type	enum		AGENT, COORDINATOR
provider_id	UUID	FK → Provider.id	Tenant scope
wallet_type	enum		PHYSICAL_CASH, E_MONEY
balance	decimal(18,2)		Integer minor units in practice
status	enum		ACTIVE, FROZEN

LiquidityRequest - *shortage request; tracks the request lifecycle*

Field	Type	Key	Notes
id	UUID	PK	Primary key
requester_id	UUID	FK (polymorphic) → Agent.id / LiquidityCoordinator.id	Who opened the request
requester_type	enum		AGENT, COORDINATOR
liquidity_type	enum		PHYSICAL_CASH, E_MONEY
provider_id	UUID	FK → Provider.id	Tenant scope
area_id	UUID	FK → Area.id	Matching scope
amount	decimal(18,2)		Requested amount
urgency	enum		LOW, MEDIUM, HIGH
status	enum		DRAFT, PENDING, OPEN, ASSIGNED,

			FULFILLED, EXPIRED, CANCELLED
expires_at	timestamp		Auto-expiry
created_at	timestamp		Record creation time
updated_at	timestamp		Last update time

LiquidityOffer — *coordinator response to a request; multiple offers may exist*

Field	Type	Key	Notes
id	UUID	PK	Primary key
request_id	UUID	FK → LiquidityRequest.id	Target request
coordinator_id	UUID	FK → LiquidityCoordinator.id	Offering coordinator
available_amount	decimal(18,2)		Amount offered
eta_minutes	int		Estimated arrival time
status	enum		SUBMITTED, WITHDRAWN, ACCEPTED, REJECTED

LiquidityAssignment — *selected offer; one active assignment per request*

Field	Type	Key	Notes
id	UUID	PK	Primary key
request_id	UUID	FK → LiquidityRequest.id, Unique*	*Unless split mode is enabled
offer_id	UUID	FK → LiquidityOffer.id	Accepted offer
assigned_to	UUID	FK → User.id	Party fulfilling the request
assigned_by	UUID	FK → User.id	Party who authorized the assignment
assigned_amount	decimal(18,2)		Locked amount
status	enum		ACTIVE, COMPLETED, CANCELLED

LiquidityTransfer — *actual value movement; must be atomic and idempotent*

Field	Type	Key	Notes
id	UUID	PK	Primary key

assignment_id	UUID	FK → LiquidityAssignment.id, Nullable	Linked assignment
from_wallet_id	UUID	FK → Wallet.id	Debited wallet
to_wallet_id	UUID	FK → Wallet.id	Credited wallet
amount	decimal(18,2)		Positive integer minor units
transfer_type	enum		COORDINATOR_SUPPLY, AGENT_SUPPLY, PROVIDER_SUPPLY
idempotency_key	varchar(64)	Unique	Prevents duplicate settlement
status	enum		PENDING, SUCCESS, FAILED, REVERSED
completed_at	timestamp	Nullable	Set on success

PointTransaction - *reward ledger; used for earned and reversed points*

Field	Type	Key	Notes
id	UUID	PK	Primary key
user_id	UUID	FK → User.id	Point owner
transfer_id	UUID	FK → LiquidityTransfer.id, Nullable	Originating transfer
activity_type	enum		EARNED, REVERSED, REDEEMED
amount	decimal(18,2)	Nullable	Related monetary value
points	int		Point delta
created_at	timestamp		Record creation time

Notification - *inbox and delivery state; non-blocking side effect*

Field	Type	Key	Notes
id	UUID	PK	Primary key
user_id	UUID	FK → User.id	Recipient
type	varchar(50)		Notification category
payload	jsonb		Structured message data
read_at	timestamp	Nullable	Set when read
status	enum		PENDING, SENT, READ

Dispute - *exception workflow; used for reversals and reviews*

Field	Type	Key	Notes
id	UUID	PK	Primary key

request_id	UUID	FK → LiquidityRequest.id, Nullable	Related request
transfer_id	UUID	FK → LiquidityTransfer.id, Nullable	Related transfer
opened_by	UUID	FK → User.id	Reporting party
reason	text		Free-text explanation
status	enum		OPEN, UNDER_REVIEW, RESOLVED, REJECTED

AuditLog — *immutable history; required for compliance and tracing*

Field	Type	Key	Notes
id	UUID	PK	Primary key
actor_id	UUID	FK → User.id, Nullable	Who performed the action
entity_type	varchar(50)		Target entity name
entity_id	UUID		Target record id
action	varchar(50)		e.g. CREATE, UPDATE, TRANSFER
before	jsonb	Nullable	Prior state snapshot
after	jsonb	Nullable	New state snapshot
created_at	timestamp		Record creation time

VerificationToken — *OTP and identity verification; supports registration and reset flows*

Field	Type	Key	Notes
id	UUID	PK	Primary key
user_id	UUID	FK → User.id	Token owner
token	varchar(255)	Unique	OTP or verification code
purpose	enum		REGISTER, PASSWORD_RESET
expires_at	timestamp		Expiry time
status	enum		ACTIVE, USED, EXPIRED

RefreshToken — *session renewal; supports secure session management*

Field	Type	Key	Notes
id	UUID	PK	Primary key

user_id	UUID	FK → User.id	Session owner
token_hash	varchar(255)	Unique	Hashed refresh token
expires_at	timestamp		Expiry time
revoked_at	timestamp	Nullable	Set when revoked

EmailNotification — *email delivery record; tracks OTP and transactional emails*

Field	Type	Key	Notes
id	UUID	PK	Primary key
user_id	UUID	FK → User.id	Recipient
template	varchar(50)	FK → NotificationTemplate.code	Template used
subject	varchar(150)		Rendered subject line
payload	jsonb		Template variables
status	enum		QUEUED, SENT, FAILED
sent_at	timestamp	Nullable	Delivery time

NotificationTemplate - *reusable email template; supports OTP and operational emails*

Field	Type	Key	Notes
id	UUID	PK	Primary key
code	varchar(50)	Unique	Template identifier
subject	varchar(150)		Default subject line
body	text		Template body (Handlebars/Pug)
channel	enum		EMAIL
status	enum		ACTIVE, INACTIVE

6.2 Entity Relationships

From	Foreign Key	To	Type
Agent	user_id	User	One-to-One
LiquidityCoordinator	user_id	User	One-to-One
Agent	provider_id	Provider	Many-to-One
Agent	area_id	Area	Many-to-One
LiquidityCoordinator	area_id	Area	Many-to-One
CoordinatorProvider	coordinator_id / provider_id	LiquidityCoordinator ↔ Provider	Many-to-Many (join)
Wallet	owner_id (polymorphic)	Agent / LiquidityCoordinator	Many-to-One

Wallet	provider_id	Provider	Many-to-One
LiquidityRequest	requester_id (polymorphic)	Agent / LiquidityCoordinator	Many-to-One
LiquidityRequest	provider_id	Provider	Many-to-One
LiquidityRequest	area_id	Area	Many-to-One
LiquidityOffer	request_id	LiquidityRequest	Many-to-One
LiquidityOffer	coordinator_id	LiquidityCoordinator	Many-to-One
LiquidityAssignment	request_id	LiquidityRequest	One-to-One
LiquidityAssignment	offer_id	LiquidityOffer	One-to-One
LiquidityAssignment	assigned_to	User	Many-to-One
LiquidityAssignment	assigned_by	User	Many-to-One
LiquidityTransfer	assignment_id	LiquidityAssignment	One-to-One
LiquidityTransfer	from_wallet_id	Wallet	Many-to-One
LiquidityTransfer	to_wallet_id	Wallet	Many-to-One
PointTransaction	user_id	User	Many-to-One
PointTransaction	transfer_id	LiquidityTransfer	Many-to-One
Notification	user_id	User	Many-to-One
Dispute	request_id	LiquidityRequest	Many-to-One
Dispute	transfer_id	LiquidityTransfer	Many-to-One
Dispute	opened_by	User	Many-to-One
AuditLog	actor_id	User	Many-to-One
VerificationToken	user_id	User	Many-to-One
RefreshToken	user_id	User	Many-to-One
EmailNotification	user_id	User	Many-to-One
EmailNotification	template	NotificationTemplate	Many-to-One

7. Core Business Rules

7.1 Allowed Liquidity Flows

Sender	Receiver	Allowed
Provider	Coordinator	Yes
Agent	Coordinator	Yes
Coordinator	Agent	Yes
Coordinator	Coordinator	Yes
Provider	Coordinator	Yes

Agent	Agent	No
-------	-------	----

7.2 Request Lifecycle

1. Request is created in DRAFT or PENDING state.
2. System validates role, tenant scope, amount, wallet availability, and expiry.
3. Request becomes OPEN and is visible to matching actors.
4. Coordinators submit offers.
5. An authorized actor accepts an offer and creates an assignment.
6. A transfer is executed and the request is marked fulfilled or partially fulfilled.
7. If no matching occurs before expiry, the request is closed as expired.

7.3 Transfer Rules

- A transfer must never debit and credit the same wallet.
- Transfer amounts must be positive integers in minor currency units.
- The source wallet must have sufficient available balance and must not be frozen.
- Transfers must use idempotency keys to prevent double settlement on retries.
- A failed transfer should not leave a partial balance update behind.

8. API Specification

All APIs are assumed to live under `/api/v1`. Access control is enforced by JWT access token plus role and tenant scope checks. Write endpoints that can be retried must accept an `Idempotency-Key` header. Email delivery for OTP and notifications is handled asynchronously through the mailer layer so the API does not block while SMTP or provider delivery is happening.

8.0 Implementation Priority

Phase	Scope
Phase 1 - Core transaction path	Registration, login, session handling, create request, submit/accept offer, execute transfer
Phase 2 - Network features	Points/incentives, notifications, dispute handling, dashboards
Phase 3 - Extensions	Redemption catalog, advanced reporting exports

8.1 Authentication APIs

Method	Endpoint	Purpose	Key Inputs
--------	----------	---------	------------

POST	/auth/register	Create a new user account and begin verification	name, email or phone, password, role, provider_id optional, area_id optional
POST	/auth/verify-otp	Verify registration or password-reset OTP	verification_token, otp_code
POST	/auth/resend-otp	Resend OTP with throttling through email or fallback channel	verification_token
POST	/auth/login	Authenticate and issue tokens	identifier, password
POST	/auth/refresh-token	Rotate access token	refresh_token
POST	/auth/logout	Revoke a session	refresh_token
POST	/auth/forgot-password	Start password reset	email or phone
POST	/auth/reset-password	Complete password reset	reset_token, new_password
GET	/auth/me	Return the current user profile	-
PATCH	/auth/me	Update self profile	name, email, phone
POST	/auth/change-password	Change password after login	current_password, new_password

8.2 Provider APIs

Method	Endpoint	Purpose	Key Inputs
POST	/providers	Create provider record	name, code, contact details
GET	/providers	List providers	filters
GET	/providers/{id}	Get one provider	-
PATCH	/providers/{id}	Update provider profile	name, contact fields, status
POST	/providers/{id}/approve	Approve provider onboarding	approval_note
POST	/providers/{id}/reject	Reject provider onboarding	rejection_reason
GET	/providers/{id}/agents	List agents under provider	pagination

GET	/providers/{id}/coordinators	List provider-linked coordinators	filters
POST	/providers/{id}/coordinators/{coordinatorId}/approve	Approve coordinator link	approval_note
POST	/providers/{id}/coordinators/{coordinatorId}/reject	Reject coordinator link	rejection_reason
GET	/providers/{id}/shortages	List shortages in provider scope	filters
GET	/providers/{id}/dashboard	Provider dashboard metrics	-
POST	/providers/{id}/wallets/supply	Supply liquidity to a coordinator	from_wallet_id, to_wallet_id, amount, transfer_type

8.3 Agent APIs

Method	Endpoint	Purpose	Key Inputs
POST	/agents/onboard	Create agent profile	user_id, provider_id, area_id, shop_name, address
GET	/agents/{id}	View agent profile	-
PATCH	/agents/{id}	Update agent profile	shop_name, address, area_id
POST	/agents/{id}/verify	Verify agent details	verification_note
GET	/agents/{id}/wallets	List agent wallets	-
POST	/agents/{id}/requests	Create liquidity request	type, amount, provider_id, area_id, urgency, notes
GET	/agents/{id}/requests	List agent requests	status filter
GET	/agents/{id}/offers	List offers related to agent requests	status filter
POST	/agents/{id}/supply	Offer excess liquidity to coordinator	wallet_id, amount, coordinator_id

GET	/agents/{id}/points	View points history	-
-----	---------------------	---------------------	---

8.4 Coordinator APIs

Method	Endpoint	Purpose	Key Inputs
POST	/coordinators/onboard	Create coordinator profile	user_id, area_id, name
GET	/coordinators/{id}	View coordinator profile	-
PATCH	/coordinators/{id}	Update coordinator profile	name, area_id
POST	/coordinators/{id}/apply-provider-link	Request provider association	provider_id, message
GET	/coordinators/{id}/providers	View approved and pending provider links	-
GET	/coordinators/{id}/available-requests	Search open requests	filters
POST	/coordinators/{id}/offers	Submit an offer	request_id, available_amount, eta_minutes
PATCH	/coordinators/{id}/offers/{offerId}/withdraw	Withdraw unaccepted offer	withdrawal_reason
POST	/coordinators/{id}/requests/{requestId}/accept	Accept assignment	offer_id, assigned_amount
POST	/coordinators/{id}/requests/{requestId}/decline	Decline an offer or request	reason
POST	/coordinators/{id}/transfers	Execute liquidity transfer	from_wallet_id, to_wallet_id, amount, transfer_type
GET	/coordinators/{id}/transactions	View transaction history	filters
GET	/coordinators/{id}/points	View point ledger	-

8.5 Liquidity Request APIs

Method	Endpoint	Purpose	Key Inputs
--------	----------	---------	------------

POST	/liquidity-requests	Create shortage request	requester_id, requester_type, liquidity_type, provider_id, area_id, amount, urgency, expires_at
GET	/liquidity-requests	Search requests	status, provider_id, area_id, type
GET	/liquidity-requests/{id}	View request details	-
PATCH	/liquidity-requests/{id}	Edit request before assignment	amount, notes, expiry
POST	/liquidity-requests/{id}/cancel	Cancel open request	reason
POST	/liquidity-requests/{id}/assign	Choose an offer and assign it	offer_id, assigned_amount
POST	/liquidity-requests/{id}/reopen	Reopen request when policy allows	reason
GET	/liquidity-requests/{id}/offers	List offers for request	-

8.6 Offers, Wallets, Transfers, Points, and Support APIs

Method	Endpoint	Purpose	Key Inputs
POST	/liquidity-offers	Submit offer	request_id, coordinator_id, available_amount, eta_minutes, note
PATCH	/liquidity-offers/{id}	Update offer before acceptance	available_amount, eta_minutes, note
POST	/liquidity-offers/{id}/withdraw	Withdraw offer	reason
POST	/liquidity-offers/{id}/accept	Accept offer	assigned_amount
POST	/liquidity-offers/{id}/reject	Reject offer	reason
POST	/wallets	Create wallet	owner_id, owner_type, provider_id, wallet_type
GET	/wallets	List wallets	filters
GET	/wallets/{id}	View wallet	-
GET	/wallets/{id}/ledger	View wallet ledger	date filters
POST	/wallets/{id}/freeze	Freeze wallet	reason

POST	/wallets/{id}/unfreeze	Unfreeze wallet	reason
POST	/wallets/{id}/adjust	Admin adjustment	amount, reason
POST	/liquidity-transfers	Initiate transfer	from_wallet_id, to_wallet_id, amount, transfer_type, request_id, assignment_id
GET	/liquidity-transfers/{id}	View transfer	-
POST	/liquidity-transfers/{id}/confirm	Confirm receipt	confirmation_note
POST	/liquidity-transfers/{id}/fail	Mark transfer failed	failure_reason
POST	/liquidity-transfers/{id}/reverse	Reverse successful transfer	reversal_reason
GET	/points/balance	Return point balance	-
GET	/points/transactions	Return point ledger	filters
POST	/points/redeem	Redeem points if enabled	catalog_item_id, points
GET	/notifications	List notifications	read status
PATCH	/notifications/{id}/read	Mark notification read	-
POST	/notifications/read-all	Mark all notifications read	-
POST	/notifications/email/test	Send a test transactional email	template_code, recipient
POST	/disputes	Open dispute	request_id, transfer_id, reason
GET	/disputes/{id}	View dispute	-
POST	/disputes/{id}/comment	Add dispute comment	comment
POST	/disputes/{id}/resolve	Resolve dispute	resolution_type, note

8.7 Canonical Payload Examples

POST /api/v1/auth/register

```
{
  "name": "Karim Rahman",
  "email": "karim@example.com",
  "phone": "+8801XXXXXXXXXX",
  "password": "StrongPassword!23",
  "role": "COORDINATOR",
  "area_id": "area_01J...",
  "provider_id": null
}
```

POST /api/v1/liquidity-requests

```
{
  "requester_id": "agent_01J...",
  "requester_type": "AGENT",
  "liquidity_type": "PHYSICAL_CASH",
  "provider_id": "prov_01J...",
  "area_id": "area_01J...",
  "amount": 100000,
  "urgency": "HIGH",
  "allow_partial": true,
  "expires_at": "2026-07-20T12:45:00Z",
  "notes": "Expected customer cash-out peak."
}
```

POST /api/v1/liquidity-transfers

Headers:

Authorization: Bearer <token>

Idempotency-Key: 01JIDEMPOTENCYKEY

```
{
  "from_wallet_id": "wallet_from_01J...",
  "to_wallet_id": "wallet_to_01J...",
  "amount": 75000,
  "transfer_type": "COORDINATOR_SUPPLY",
  "request_id": "req_01J...",
  "assignment_id": "asn_01J..."
}
```

9. Validation, Edge Cases, and Error Handling

Code	When it happens	Expected behavior
VALIDATION_ERROR	Missing or malformed input	Reject request and return field-level details
UNAUTHORIZED	No valid session	Require login
FORBIDDEN	Role or tenant scope is insufficient	Deny action cleanly
NOT_FOUND	Entity does not exist or is out of scope	Return safe 404

CONFLICT	Duplicate user, duplicate state transition, or concurrent acceptance	Keep the original state and reject the second attempt
INSUFFICIENT_BALANCE	Wallet cannot cover amount	Reject and show available balance
REQUEST_ALREADY_ASSIGNED	Another party already assigned the request	Only first assignment wins
REQUEST_EXPIRED	Current time is past expiry	Prevent new assignment or transfer
WALLET_FROZEN	Wallet is frozen by policy or dispute	Stop settlement until resolved
IDEMPOTENCY_CONFLICT	Same key used with a different payload	Reject the retry as unsafe
TRANSIENT_PROCESSING_ERROR	External or internal temporary failure	Allow retry using same idempotency key
Scenario	Rule	System Response
Duplicate registration by email or phone	Identity fields must be unique	Reject with CONFLICT
OTP expires before verification	OTP is time-bound	Reject and allow resend after throttling
Role mismatch during onboarding	Profile must match account role	Reject with VALIDATION_ERROR
Agent tries agent-to-agent transfer	Direct agent-to-agent transfer is forbidden	Reject with FORBIDDEN
Coordinator offers above wallet balance	Offer cannot exceed available balance	Reject with INSUFFICIENT_BALANCE
Offer accepted after request expiry	Expired request cannot be assigned	Reject with REQUEST_EXPIRED
Concurrent offer acceptance	Only one active assignment per request	First transaction wins; later calls fail with CONFLICT
Transfer retried after timeout	Must be idempotent	Return the original result if payload matches
Debit succeeds but credit fails	Transfer must be atomic	Rollback or mark failed without partial balance corruption
Wallet frozen during pending assignment	Frozen wallets cannot move funds	Pause workflow and notify stakeholders
Points awarded then transfer reversed	Rewards must reflect net settled value	Create compensating point transaction
Provider rejects coordinator link	No provider-specific actions allowed	Disable provider-scope actions immediately
Partial fulfillment requested	Allowed only if policy enables it	Track fulfilled and remaining amounts
Request cancelled after assignment	Cancellation is blocked once transfer starts	Reject late cancellation attempts

Notification delivery fails	Notifications are non-blocking	Persist and retry asynchronously
Amount is zero or negative	Transfer amount must be positive	Reject immediately
Wallet type mismatch	Cash and e-money are separate wallet types	Reject unless a conversion workflow exists
Provider scope not approved	Coordinator cannot act for unapproved provider	Reject with FORBIDDEN
System restart mid-processing	Processing must be resumable	Continue safely from stored job state

10. Security and Non-Functional Requirements

- HTTPS only for all traffic.
- JWT access tokens with refresh rotation.
- Role-based and tenant-based access control on every endpoint.
- Rate limiting for registration, OTP, login, and search endpoints.
- Audit logs for every state change, transfer, and reversal.
- Integer-only monetary values to avoid rounding errors.
- Correlation IDs across API, queue, and worker flows.
- Wallet and point ledgers must be reconcilable from transaction history.
- Sensitive personal data should be masked in logs and dashboards.
- Provider-specific data must not leak across tenant boundaries.

11. Suggested NestJS Modules

- auth: registration, login, OTP, password reset, refresh
- users: global profile and role handling
- providers: provider records, approvals, dashboards
- agents: agent onboarding and request creation
- coordinators: coordinator onboarding, provider links, matching
- wallets: balances, ledger, freeze, adjustment
- mailer: OTP delivery, transactional email templates, SMTP integration, email logs
- liquidity-request: request lifecycle and search
- liquidity-offer: offers and acceptance
- liquidity-transfer: atomic settlement and reversal
- points: reward ledger and redemption
- notifications: inbox, email triggers, and retry delivery
- transactions: reconciliation and audit exports

12. Conclusion

This proposal presents LiquidityOS as a complete implementation plan for a NestJS, PostgreSQL, and TypeORM stack. It begins with the business need, defines the platform response, identifies the actors and data model, specifies the core APIs, and closes the major edge cases so engineering and QA can implement against a shared source of truth.